

---

# **SWS3009 – Part B**

## **Robotics / TeleOperation**

---

### **Section 3: Arduino Mega**

---

# Outline

## ■ **Arduino Basics**

- ❑ Introduction
- ❑ Programming
- ❑ External components demonstration

---

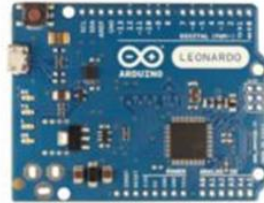
# ARDUINO BASICS

---

# Arduino Family says Hi!



Arduino Uno



Arduino Leonardo



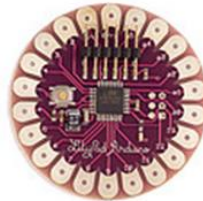
Arduino Ethernet



Arduino Pro



Arduino Mega 2560



Arduino LilyPad



Arduino BT



Arduino Nano



Arduino Mega ADK



Arduino Fio

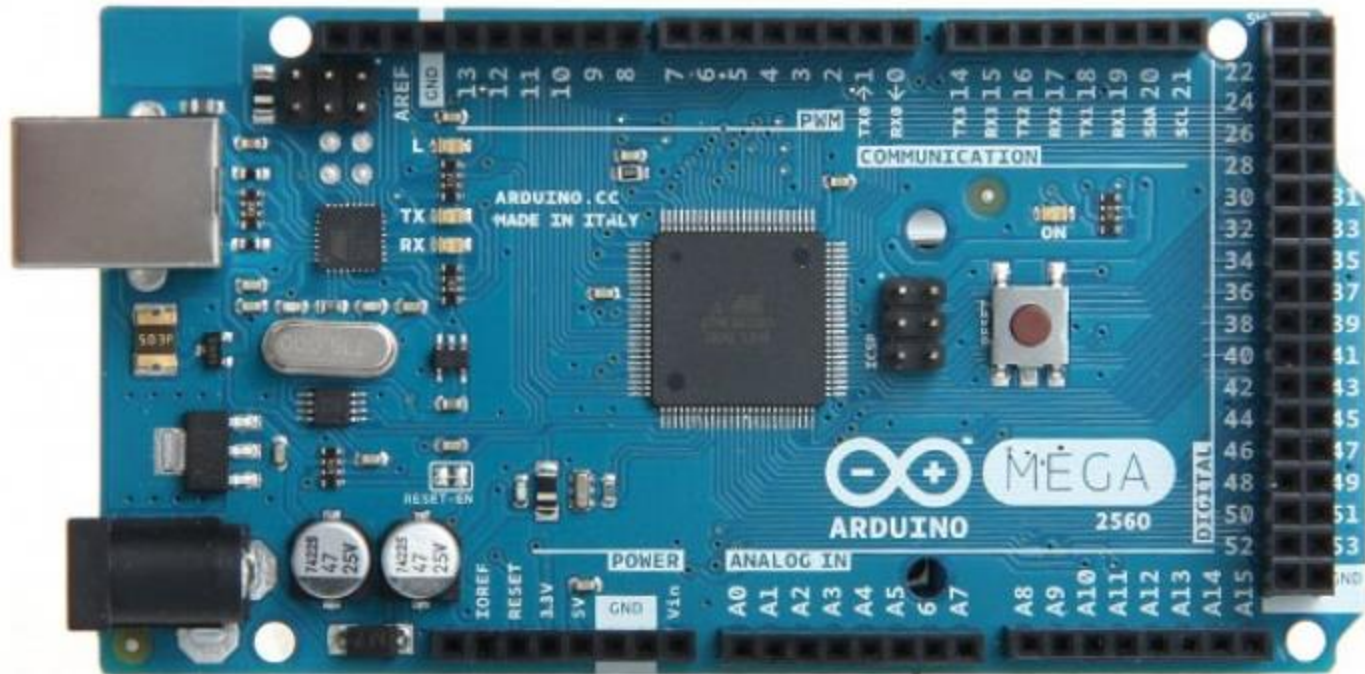


USB/Serial Light Adapter



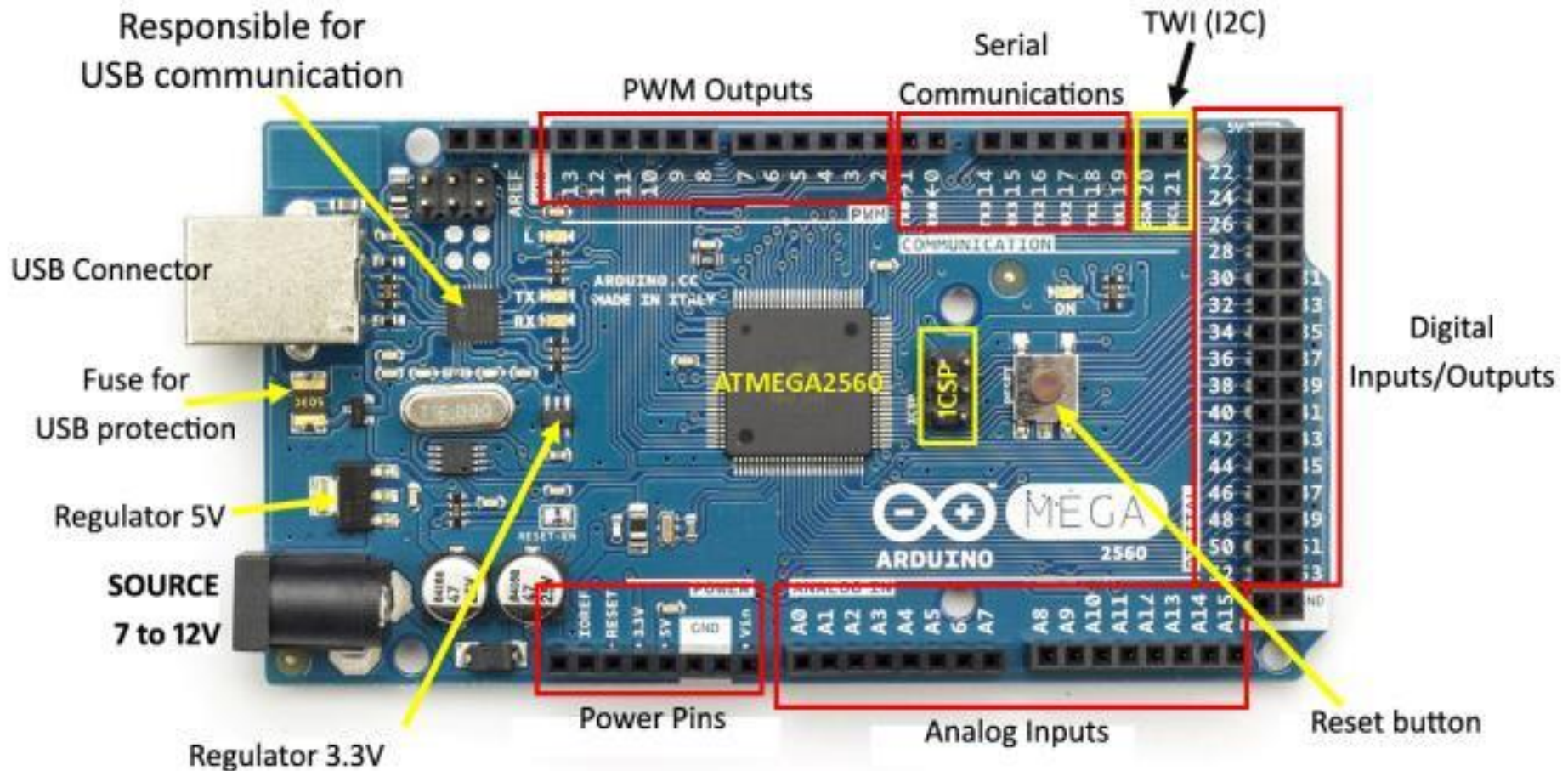
Arduino Mini

# Our Arduino....



- **Arduino Mega 2560 (Revision 3)**
  - ❑ Note that the reset button may be located differently (e.g. corner of the board)

# Arduino Mega 2560 - Layout



---

# Arduino Mega: Basic Facts

- Microcontroller Board
    - ❑ Based on ATmega2560
  - USB connection
    - ❑ Alternative Powering Option: Power Jack
  - Input / Output Capabilities
    - ❑ 54 digital input/output pins
      - 14 can be used for **PWM** output (more later)
    - ❑ 16 analog inputs
      - Digitized to 10 bits (i.e. 1024 levels)
    - ❑ 4 UART (Serial Ports)
    - ❑ 16MHz Crystal Oscillator (Clock Signal)
-

# Breadboards... how do they work?

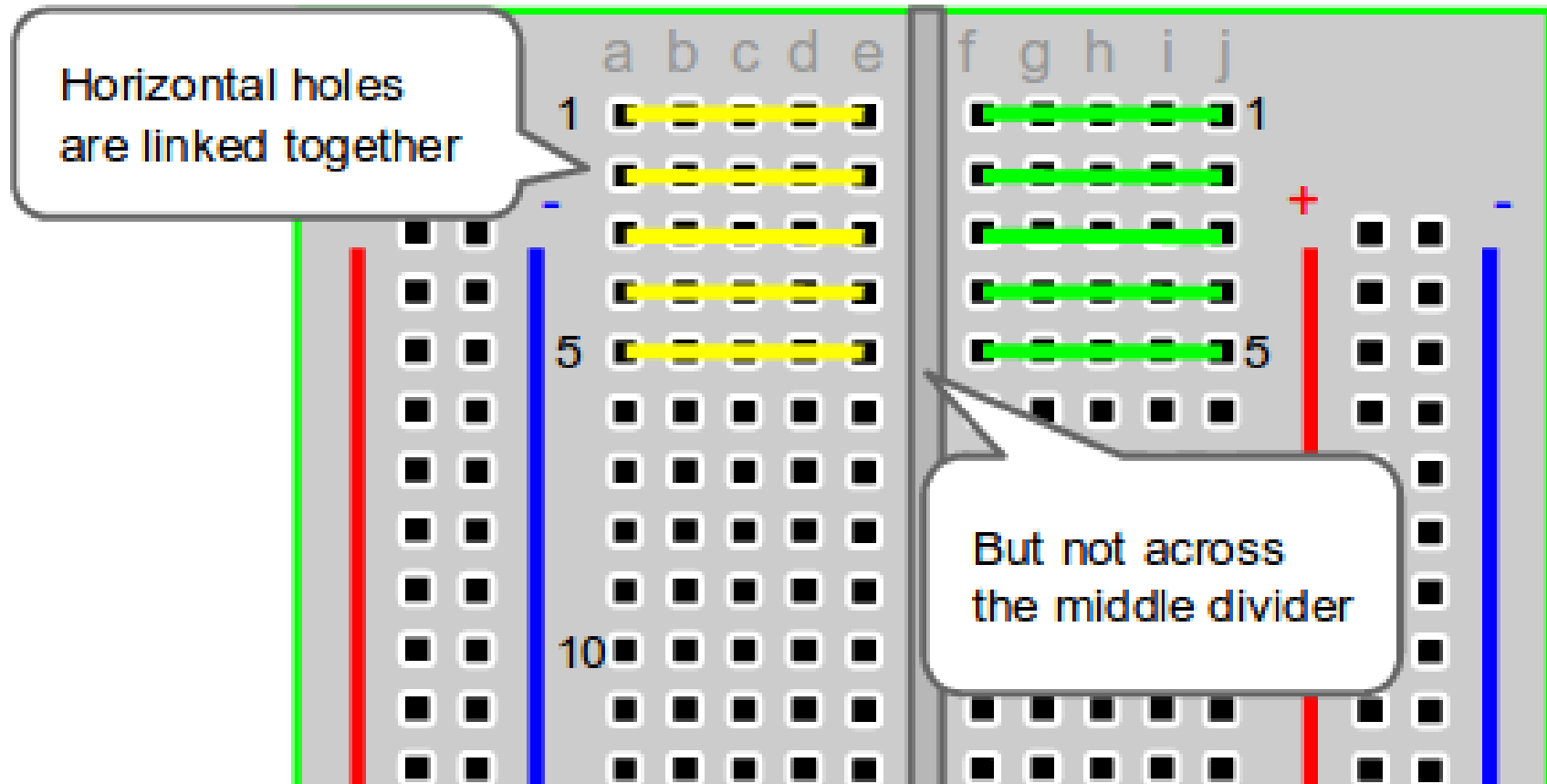


Image source: <https://computers.tutsplus.com/tutorials/how-to-use-a-breadboard-and-build-a-led-circuit--mac-54746>

# Breadboards... how do they work?

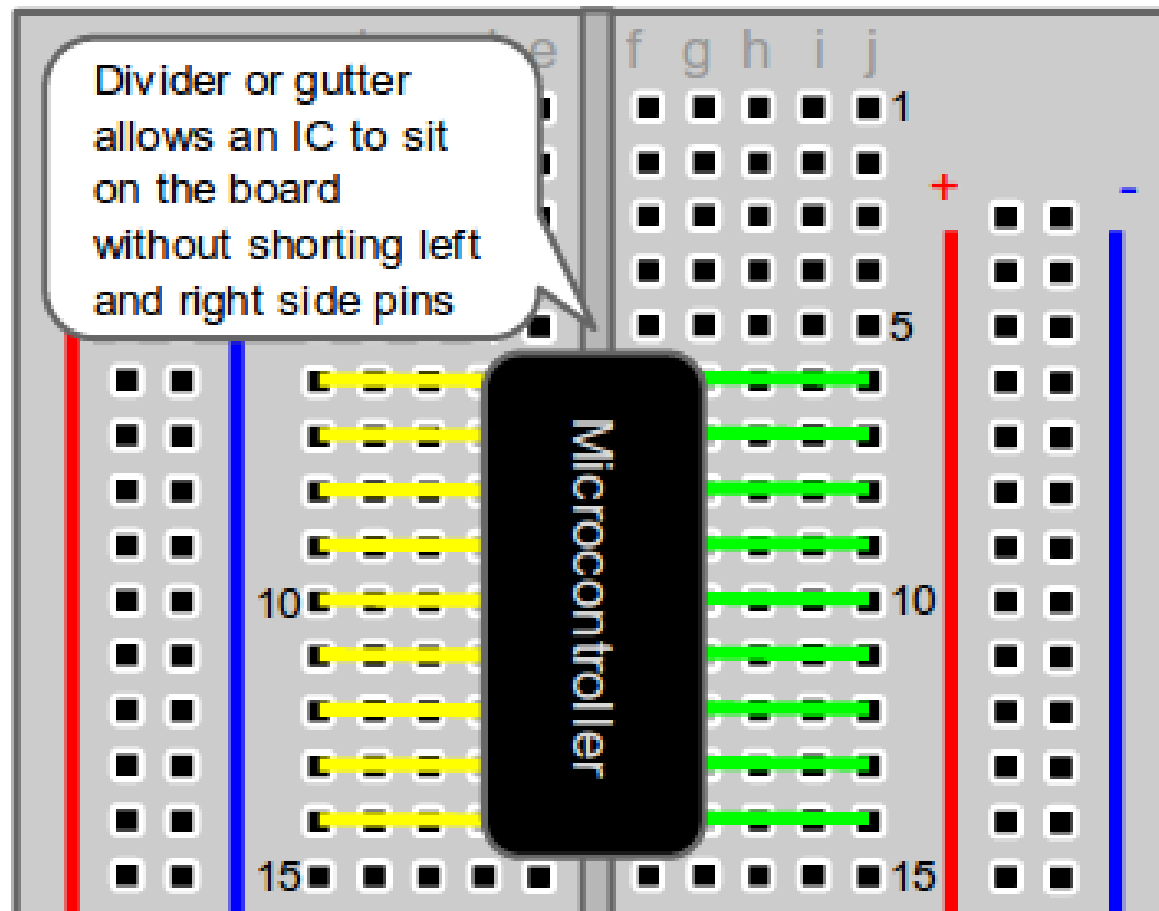


Image source: <https://computers.tutsplus.com/tutorials/how-to-use-a-breadboard-and-build-a-led-circuit--mac-54746>

# Breadboards... how do they work?

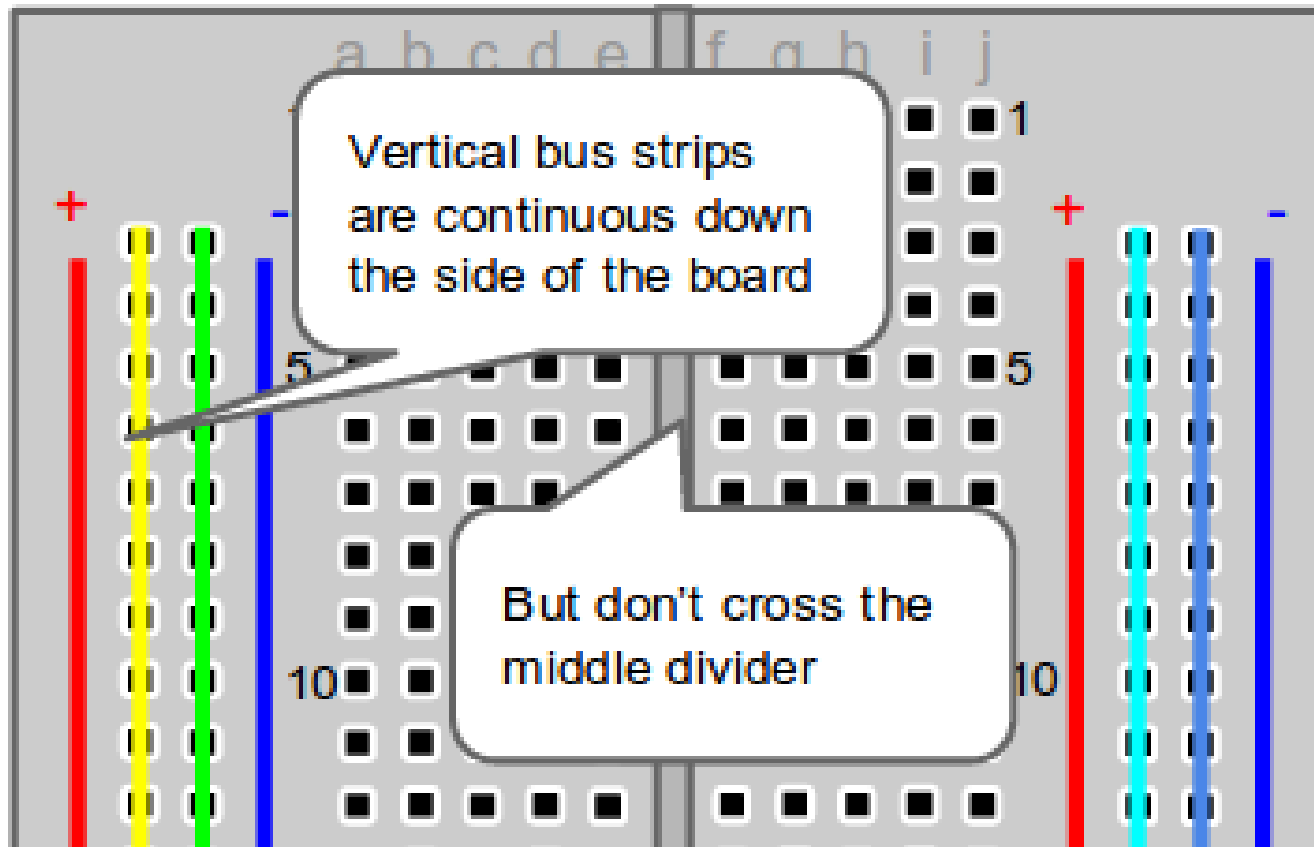
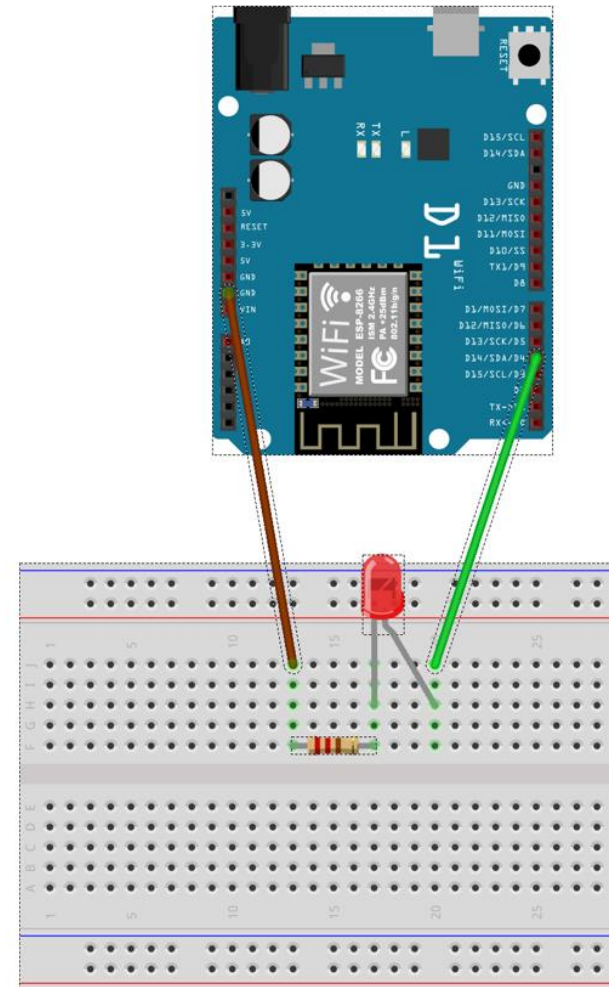
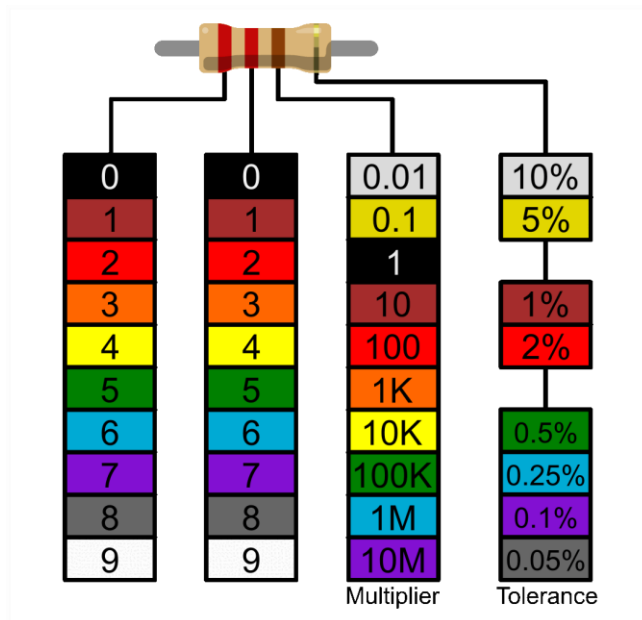


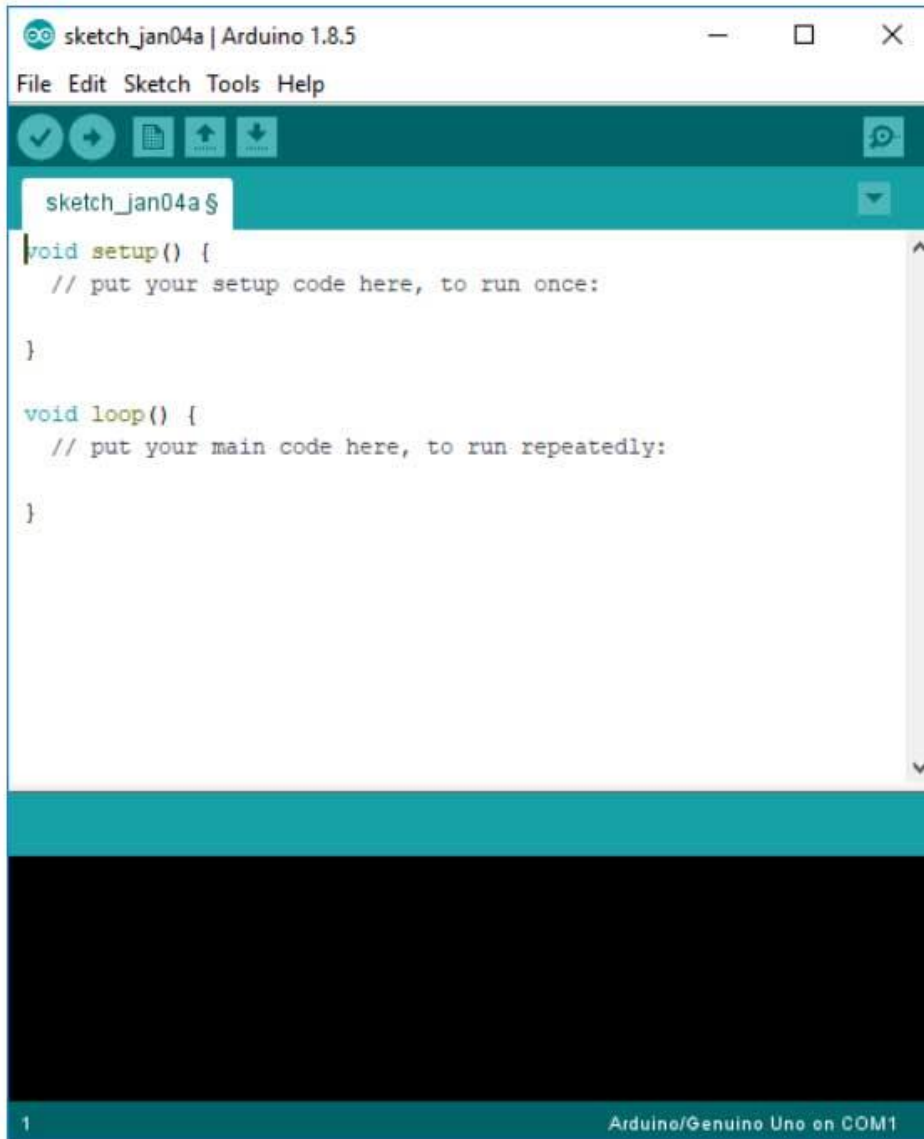
Image source: <https://computers.tutsplus.com/tutorials/how-to-use-a-breadboard-and-build-a-led-circuit--mac-54746>

# LED Task

Use a 220 Ohm resistor



# Introducing **Arduino IDE**



- Arguably the main reason for Arduino's popularity
- Minimalistic but beginner friendly and reasonably powerful

# Programming on **Arduino**

## ■ Programming Language:

- ❑ Subset of **C/C++**
- ❑ Limited set of **C/C++** libraries
- ❑ Additional set of Arduino specific calls

- Online Reference at

<https://www.arduino.cc/en/Reference/HomePage>

## ■ Program structure:

- ❑ Minimally need the `setup()` and `loop()` functions
- ❑ Additional functions can be declared

Global Variables

`void setup()`

`void loop()`

# Blink: An example program

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
  
    digitalWrite(13, LOW);  
    delay(100);  
}
```

- **pinMode( pin, mode )**
  - pin = pin number
  - mode = { INPUT, OUTPUT }
- **digitalWrite( pin, value )**
  - pin = pin number
  - value = 0 / 1 (use LOW / HIGH constant!)
- **delay( ms )**
  - ms = millisecond

---

## **Question:** Let's Blink Differently

- How do we change the blinking patterns as following:
  - **A:** Blink twice in a second
  - **B:** Blink short-short-long
  - **C:** Blink randomly
    - (Hint: Look for the correct library call in Arduino)
-

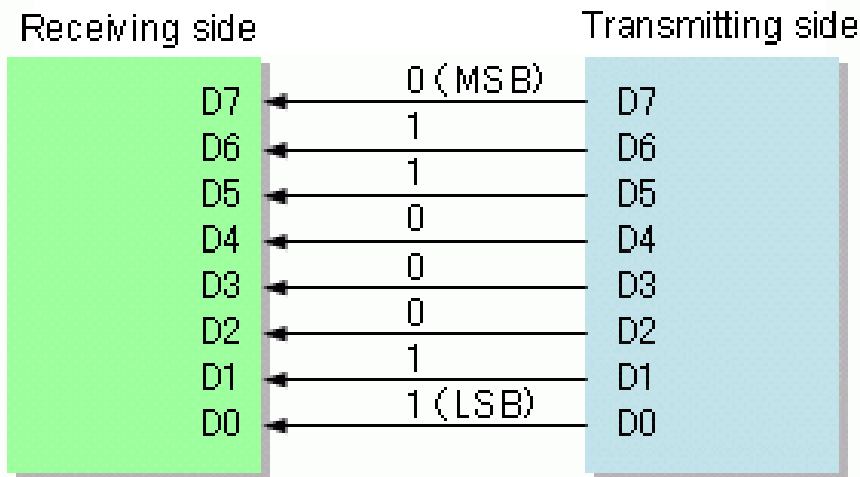
---

# Communication

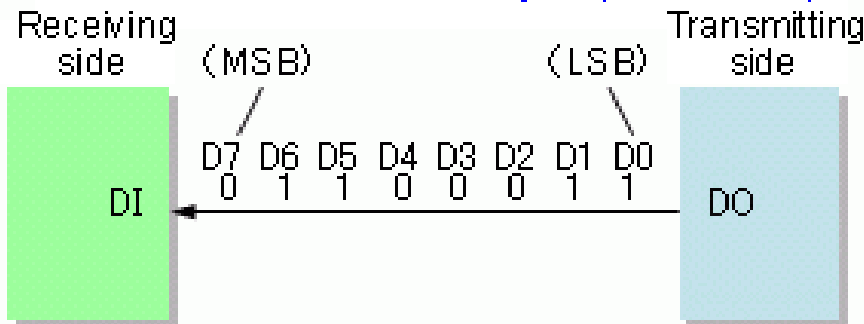
- Since we cannot easily debug the code on the Arduino directly
  - we will use **basic serial communication** as a "debugging" mechanism

# Serial Communication: Idea

## Parallel interface example



## Serial interface example (MSB first)



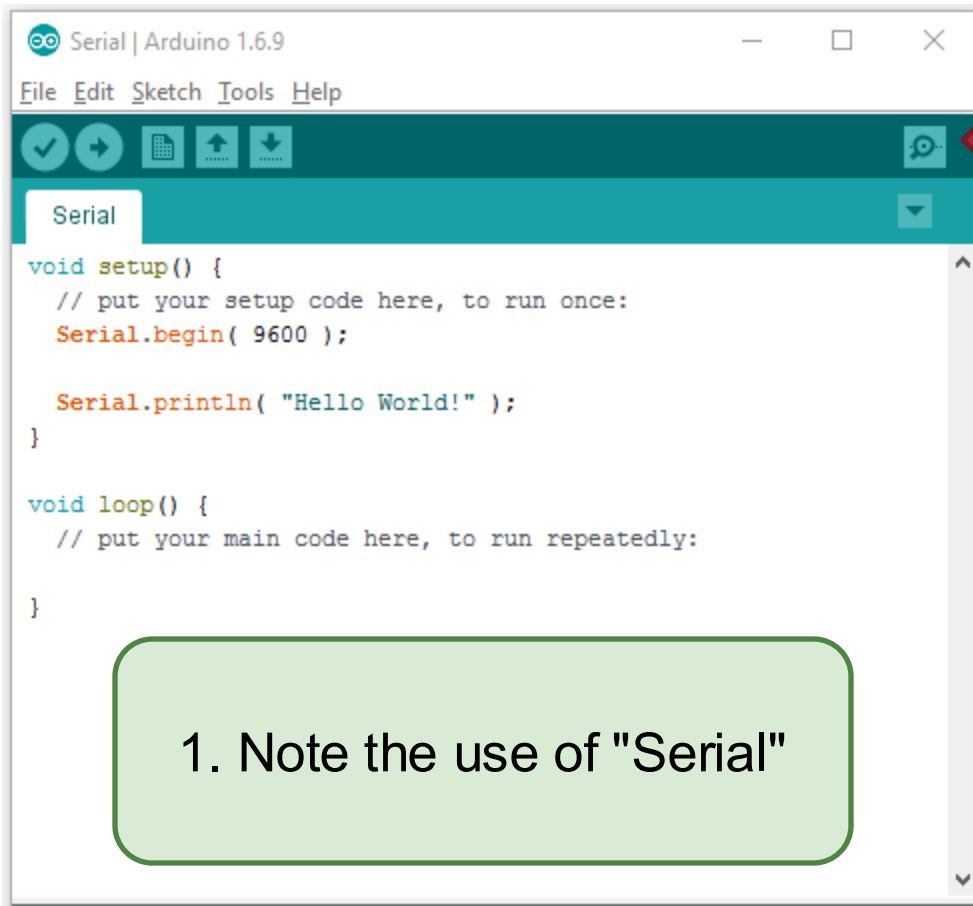
- Serial Communication is a very old idea
- Still commonly supported:
  - ❑ Only need **2 wires** for two way communication!

---

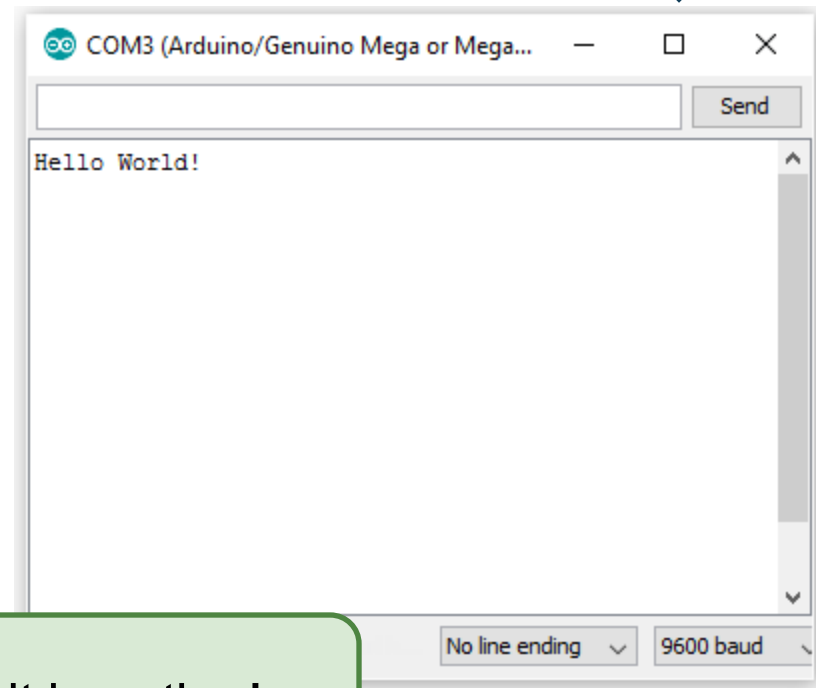
# Serial Communication: **Arduino Mega**

- **Arduino Mega support 4 sets of serial communication** (two ways each):
    - ❑ **Serial 0:** Through the USB connection or Pin 0 (Receive RX) + Pin 1 (Transmit TX)
    - ❑ **Serial 1:** Pin 19 (RX) + Pin 18 (TX),
    - ❑ **Serial 2:** Pin 17 (RX) + Pin 16 (TX)
    - ❑ **Serial 3:** Pin 15 (RX) + Pin 14 (TX)
  - **Be careful:** Check the operating voltage of the device you want to communicate with:
    - ❑ **Arduino Mega operates at 5 volt**
-

# Serial Communication: Code



2. Click ( or press Ctrl-Shift-M") to open the serial monitor

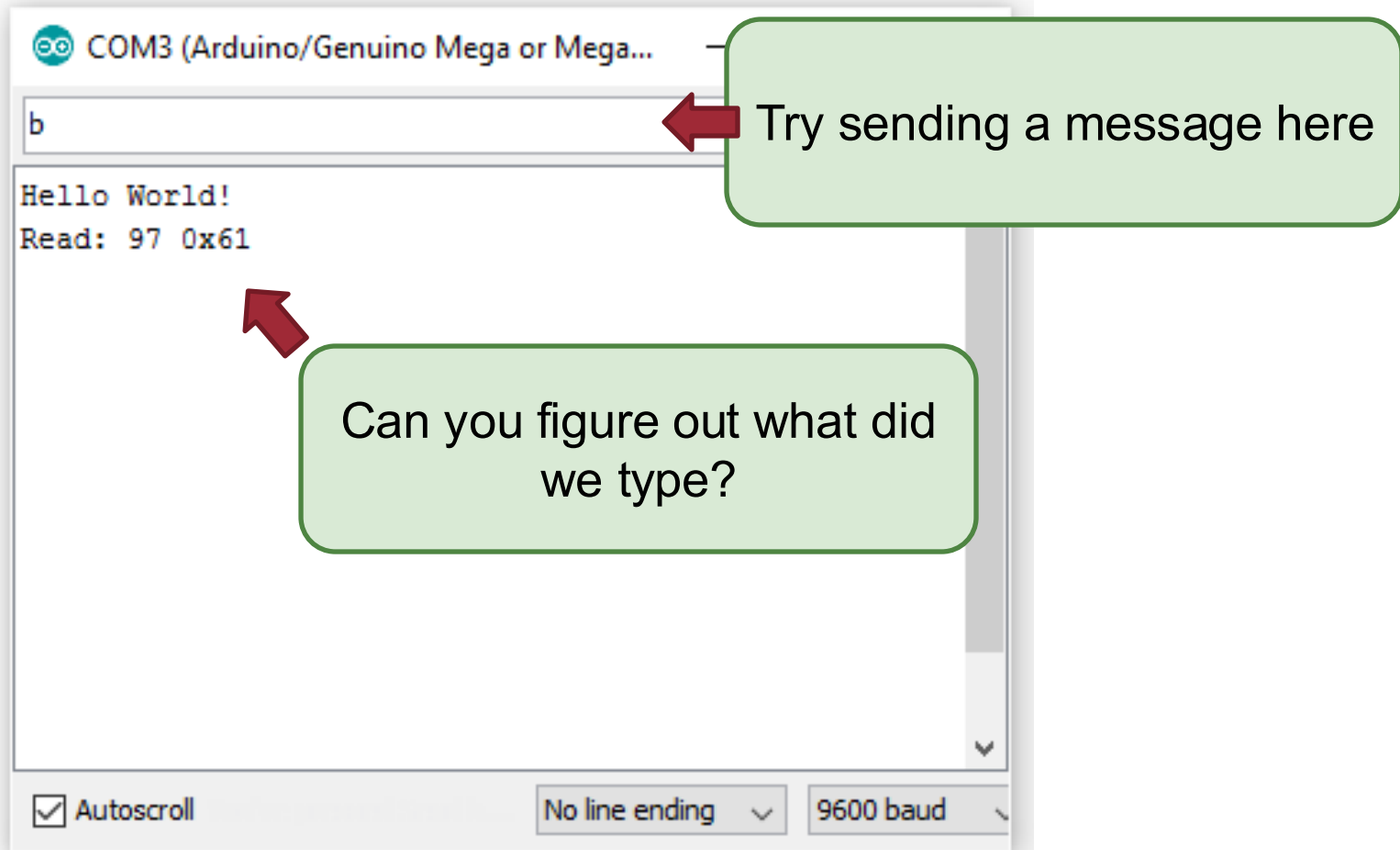


3. Let's see it in action!

# Serial Communication: Code 2

```
void setup() {  
  
    Serial.begin( 9600 );  
    Serial.println( "Hello World!" );  
  
}  
  
void loop() {  
  
    if (Serial.available()) {  
        int inByte = Serial.read();  
        Serial.print("Read: ");  
        Serial.print(inByte);  
        Serial.print(" 0x");  
        Serial.println(inByte, HEX);  
    }  
  
}
```

# Serial Communication: Code 2



---

# **EXTERNAL COMPONENTS**

---

---

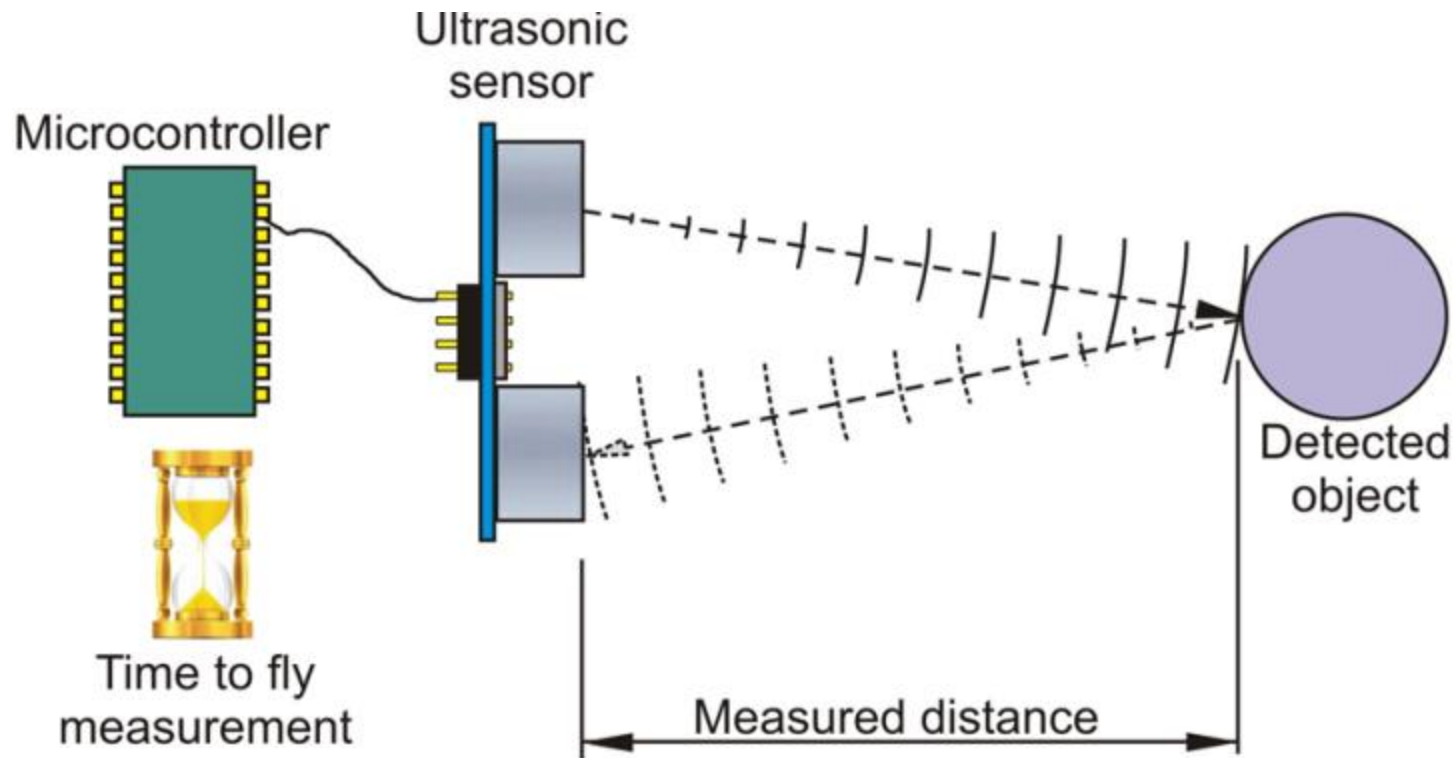
# Interfacing with Components

- Arduino can interface with many electronic components

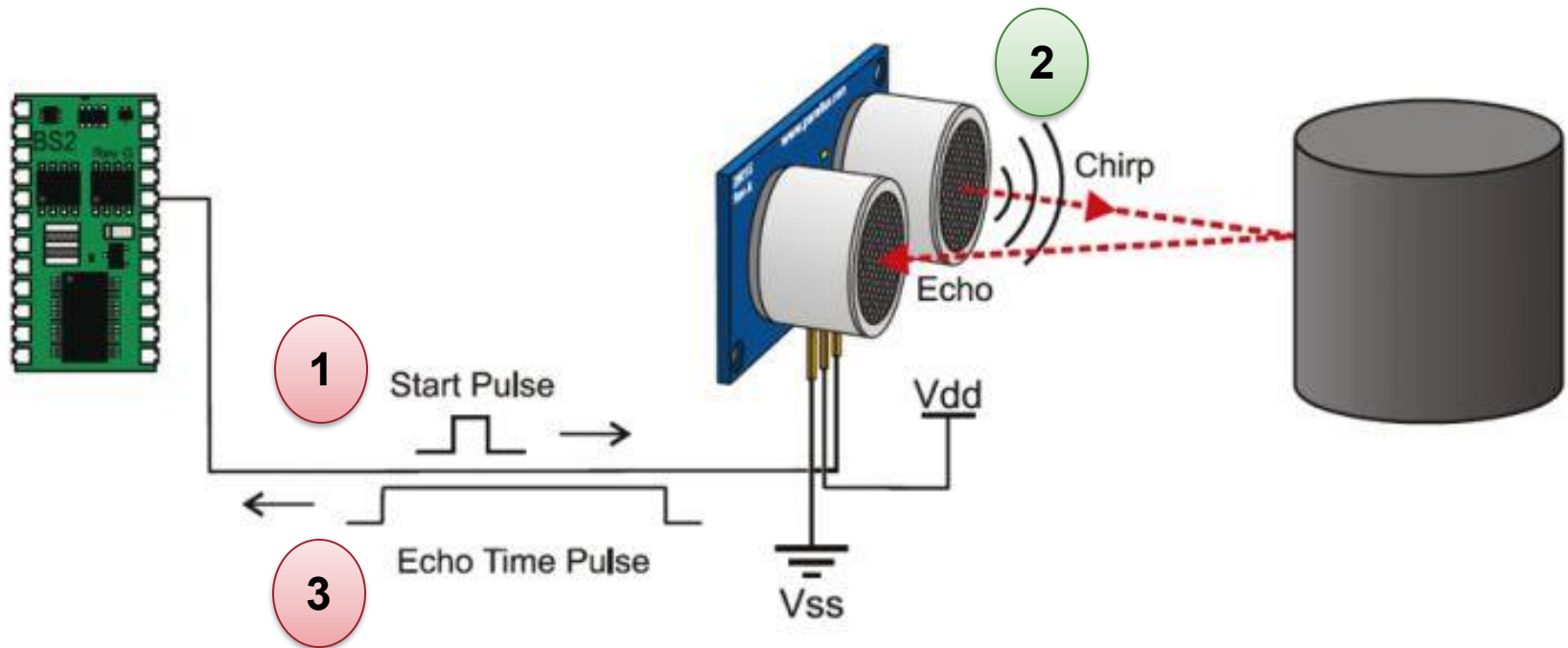
- **Common steps:**

1. Connect the electronic components to the correct pin(s)
    - Usually wiring diagram is provided
  2. Write code to interact with the component:
    - Sometimes, libraries are provided together with the components
-

# Ultrasonic Sensor - Principle

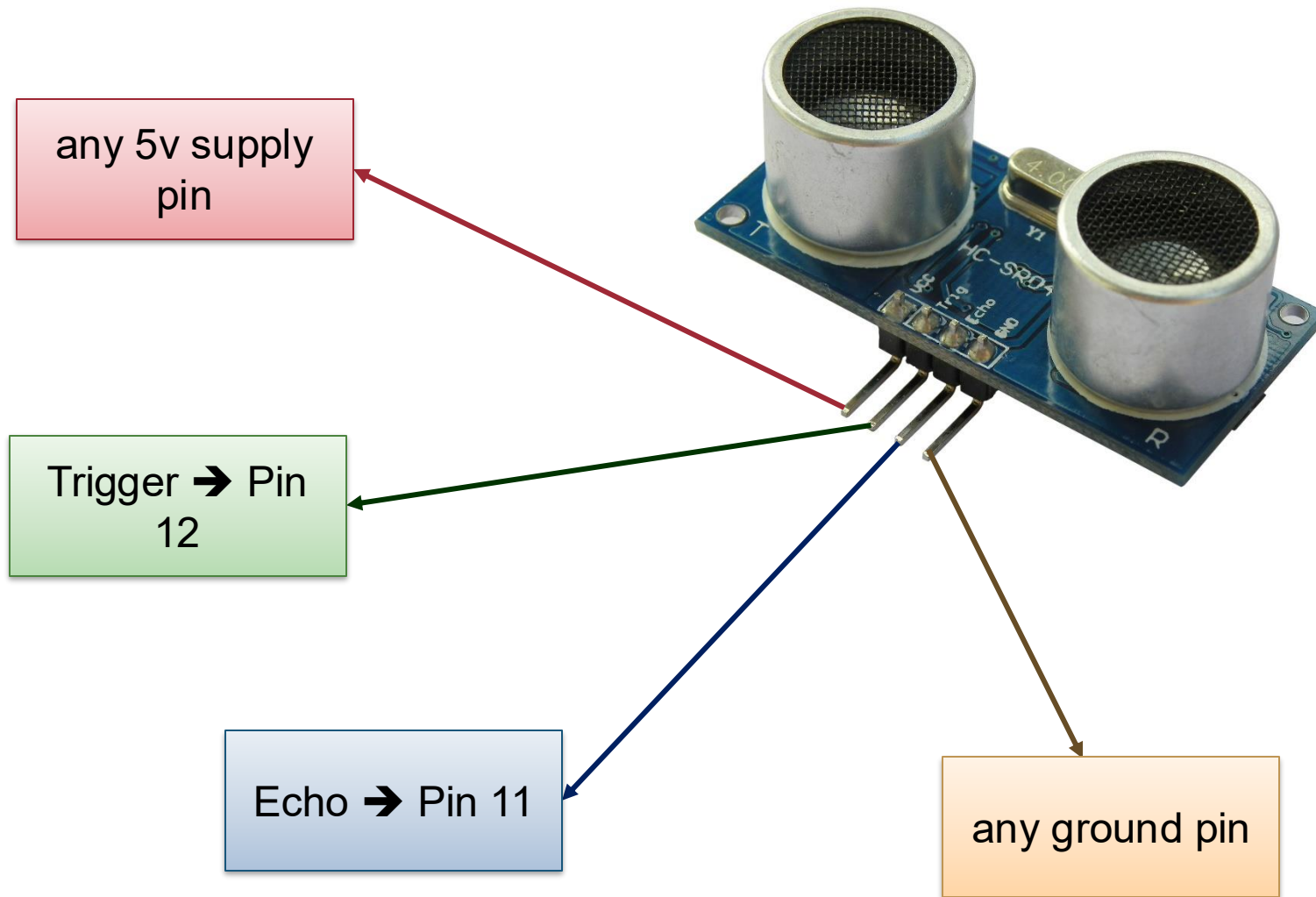


# Ultrasonic Sonic - Implementation

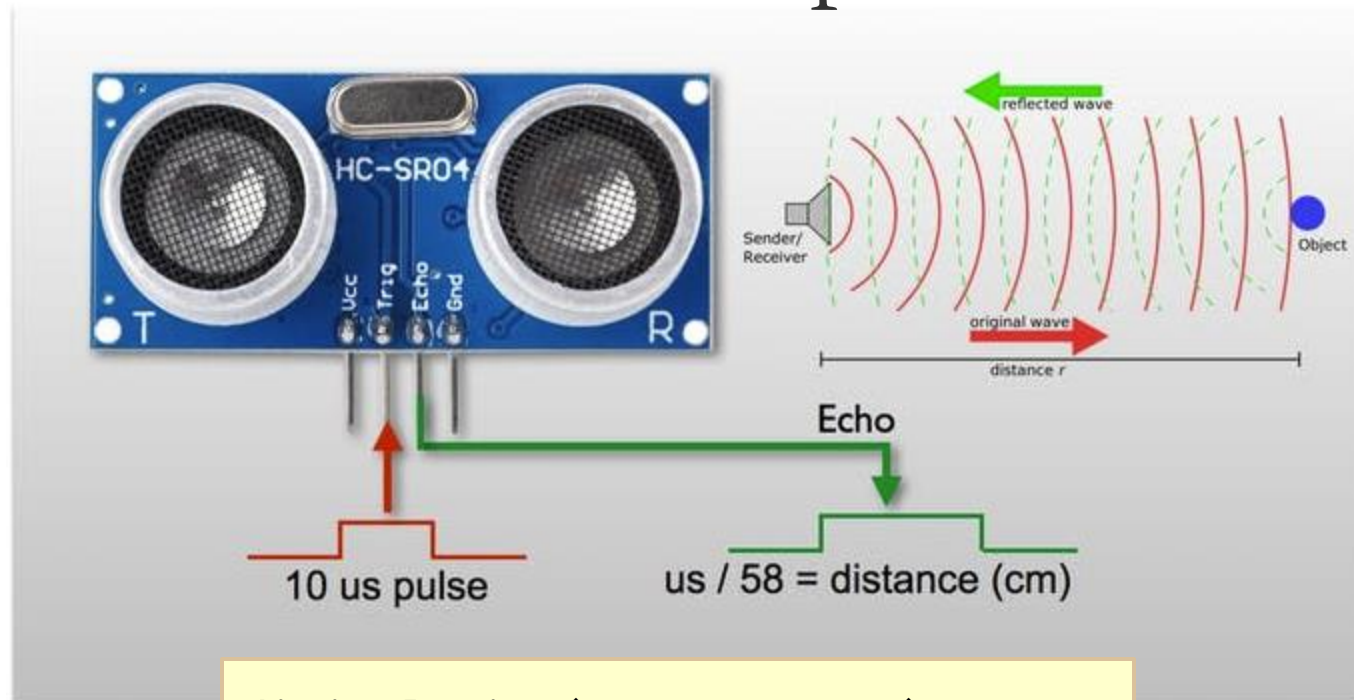


- Given **Echo Time Pulse** (i.e. time difference between Chirp and Echo), how do we get the distance, **D**?

# Ultrasonic Sensor: Connection



# Ultrasonic Sensor: Operation



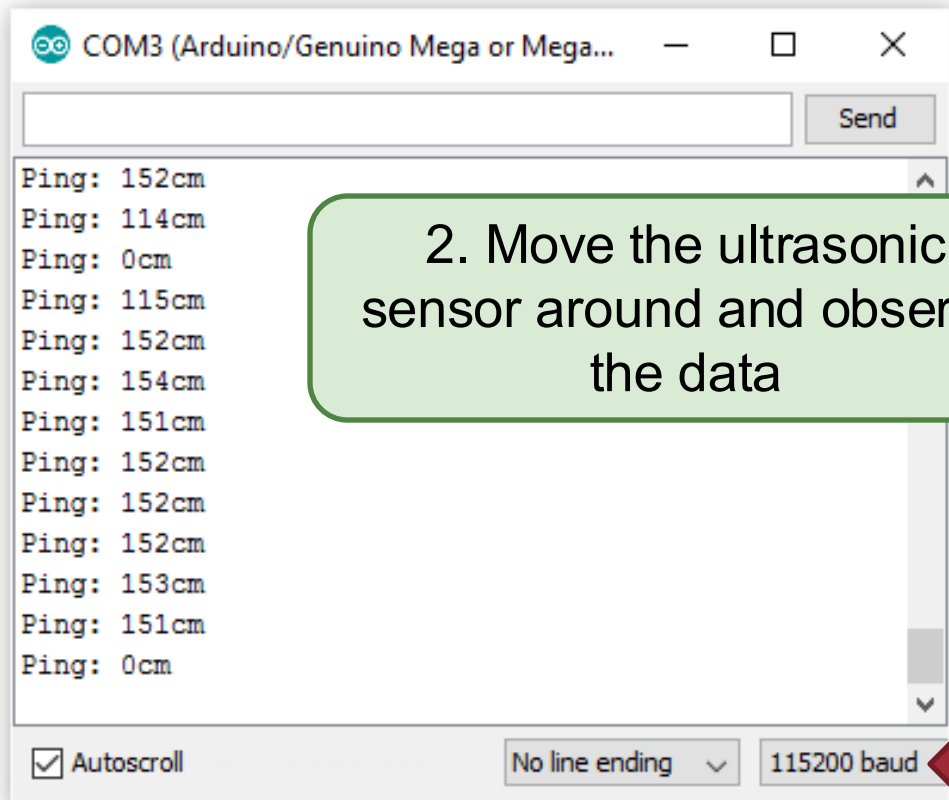
```
digitalWrite(TRIGGER, LOW);  
delayMicroseconds(2);  
digitalWrite(TRIGGER, HIGH);  
delayMicroseconds(10);  
digitalWrite(TRIGGER, LOW);  
duration = pulseIn(ECHO, HIGH);  
cm = duration / 58 ;
```

---

# Ultrasonic Sensor: Adding Libraries

- In this case, the ultrasonic sensor manufacturer provided sample library to use the sensor in a painless way
  - In Arduino, select Library Manager > Search for “NewPing by Tim Eckel” and click Install.
  - [For exploration] You can take a look in the library code to see the details, e.g. how to trigger a pulse, how to measure echo time pulse, etc
-

# Ultrasonic Sensor: Code Example



2. Move the ultrasonic sensor around and observe the data

1. Change the baud rate

- Understand the code:
  - ❑ Especially on how to define the pins used correctly

---

# DC MOTOR

---

# DC Motor: **Speed** and **Direction**

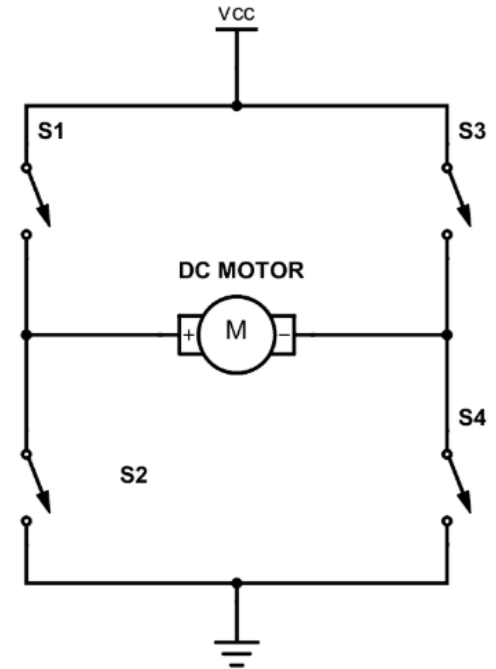
50% duty cycle



75% duty cycle



25% duty cycle



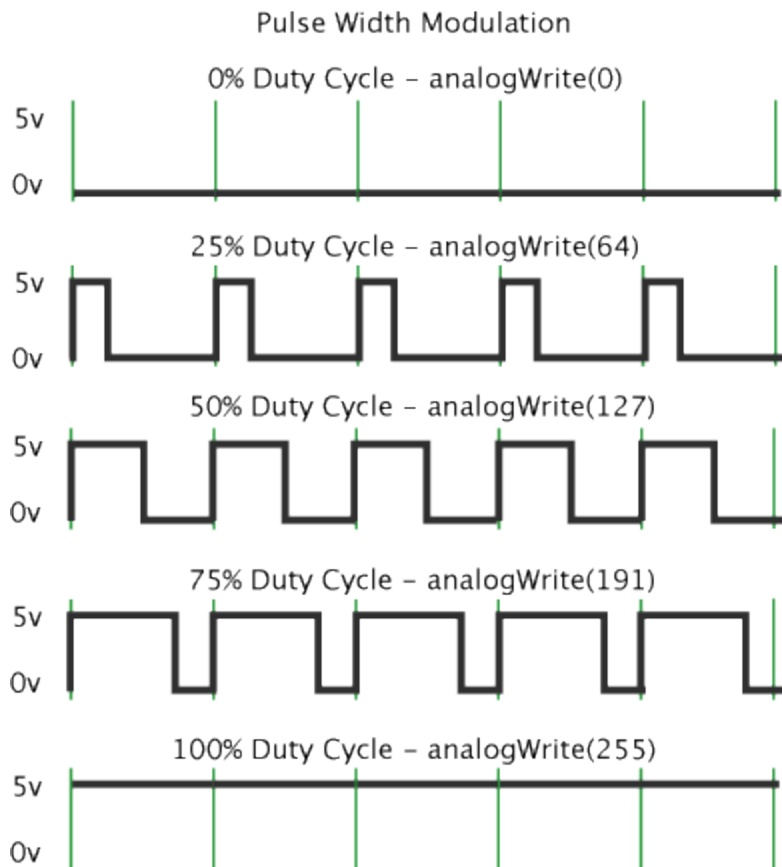
1. Turning speed is control by **PWM**
2. Turning direction is handled by **H-Bridge**

---

# Controlling the Speed of Motor

- Intuitively, we can control the speed of motor by varying the voltage supply
  - So, if 5v == maximum speed, supplying 2v should give us a slower speed...
  - The idea is correct, but there is a problem...
-

# Pulse Width Modulation (**PWM**)



- Sometimes we need to have **analog output**:
  - ❑ e.g. varying amount of power to the motor will modify the speed
- PWM enables us to simulate analog output by switching on/off rapidly on an digital output channel
- Pin 2 to 13, 44 to 46 supports PWM

---

# Exercise: PWM for LED

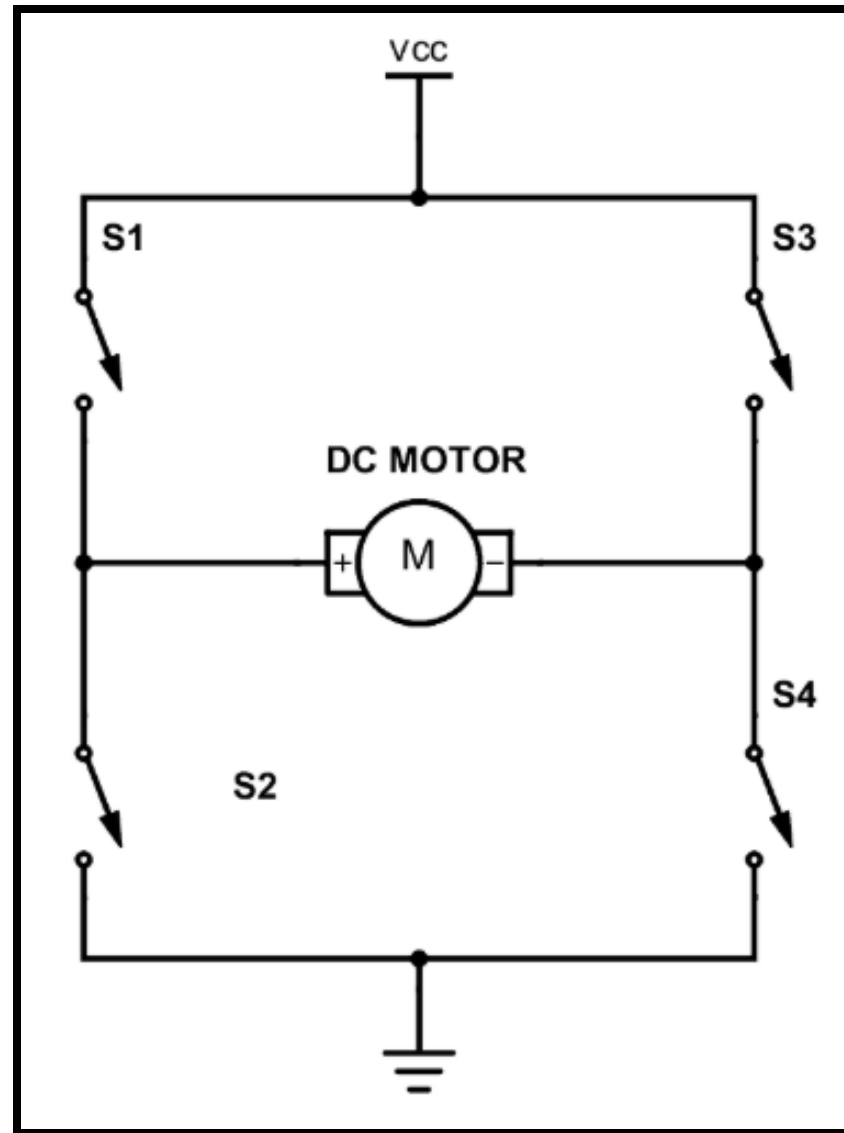
- Let's try to use PWM to control the brightness of the LED
  - Conveniently, Pin 13 supports PWM!
  - See code `Blink_PWM.ino`
- **[Additional Challenge]** Write a program to control the brightness of LED using Serial Communication

---

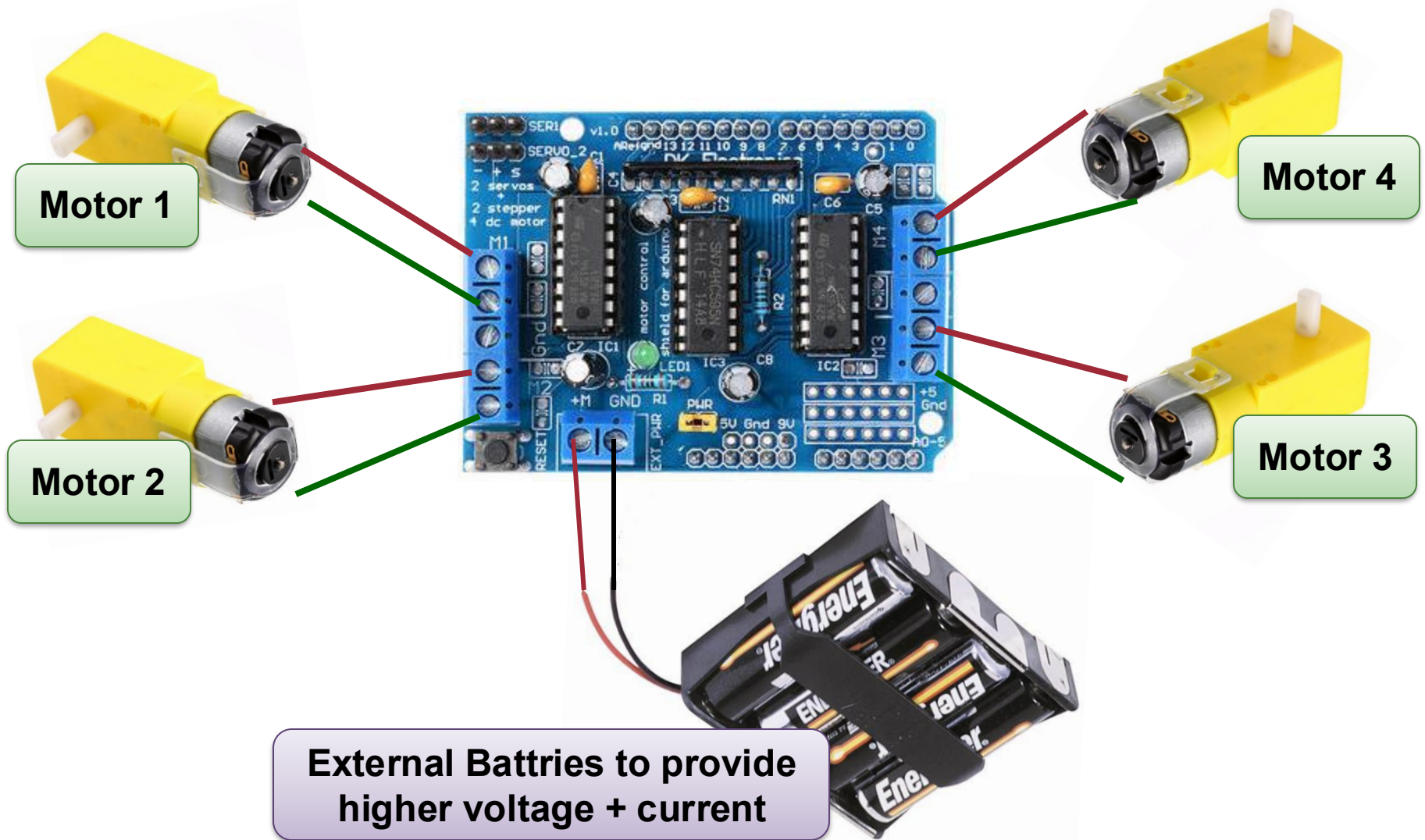
# Controlling the Direction of Motor

- H-Bridge involve multiple electronic components (e.g. transistor, resistor etc)
  - Imagine the messiness if we need to control 2-4 wheels
  - **Motor Driver / Motor Shield** is a commonly seen integrated component that provide H-bridge functionality
  - We use the Dual L293D Motor Driver Arduino Shield:
    - Can control up to 4 motors!
-

# Controlling the Direction of Motor



# Dual L293D Motor Shield



# Motor Shield: Library

- Basic functionality is provided by the "Adafruit Basic Motor Shield". Install using Arduino library manager.

```
#include <AFMotor.h>

void setup() {
  // put your setup code here, to run once:
  AF_DCMotor motor(4); // The motor number, i.e. 1, 2, 3 or 4

  motor.setSpeed( 200 );

  motor.run( FORWARD );
  delay(1000);

  motor.run( BACKWARD );
  delay(1000);

  motor.run( RELEASE );
  delay(1000);
}
```

---

Excuse me! Excuse me!

# INTERRUPT HANDLING

# Motivation

- Imagine your Arduino is running some code:
  - However, if the external emergency stop button is pressed, you are supposed to stop processing immediately
- Main question is:
  - How do we handle external event?
- Possible solution:
  - Keep checking whether the event occurs, BUT this is not very efficient (time is wasted)



# Motivation

```
void setup() {  
    Serial.begin(9600);  
}  
  
void loop() {  
    Serial.print("Working hard...");  
    delay(1000); //Do some work  
    Serial.println("done.");  
}
```



# Interrupt: **What is it?**

- Most processors provide alternative mechanisms to handle these events:
  - e.g. **interrupt**
- When the external event happens:
  1. A **signal** is generated
  2. The processor **automatically execute** code to handle the event:
    - The code is commonly known as **interrupt handler**
  3. After the event is handled, the processor **resume the work** it was performing prior to the event

# Interrupt: Arduino Mega

- Interrupt handler in Arduino Mega is known as **Interrupt Service Routine (ISR)**
- There are 6 digital pins, each given a specific interrupt number:

| Digital Pin Number | Interrupt Number |
|--------------------|------------------|
| 2                  | 0                |
| 3                  | 1                |
| 21                 | 2                |
| 20                 | 3                |
| 19                 | 4                |
| 18                 | 5                |

Note: the Arduino function ***digitalPinToInterrupt()*** helps to convert pin number to interrupt number

# Interrupt In Action: Code (1/2)

See "ISR-Simple.ino"

```
const byte ledPin = 13;
const byte interruptPin = 2;
volatile byte state = HIGH;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(interruptPin, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(interruptPin),
    blink, CHANGE );

  digitalWrite(ledPin, state);
  Serial.begin(9600);
}
```

# Interrupt In Action: Code (2/2)

```
void loop() {  
    Serial.print("Working hard...");  
    delay(1000);  
    Serial.print("done. State = ");  
    Serial.println(state);  
}  
  
void blink() {  
    state = !state;  
    digitalWrite(ledPin, state);  
}
```

# Interrupt In Action: **Connection**

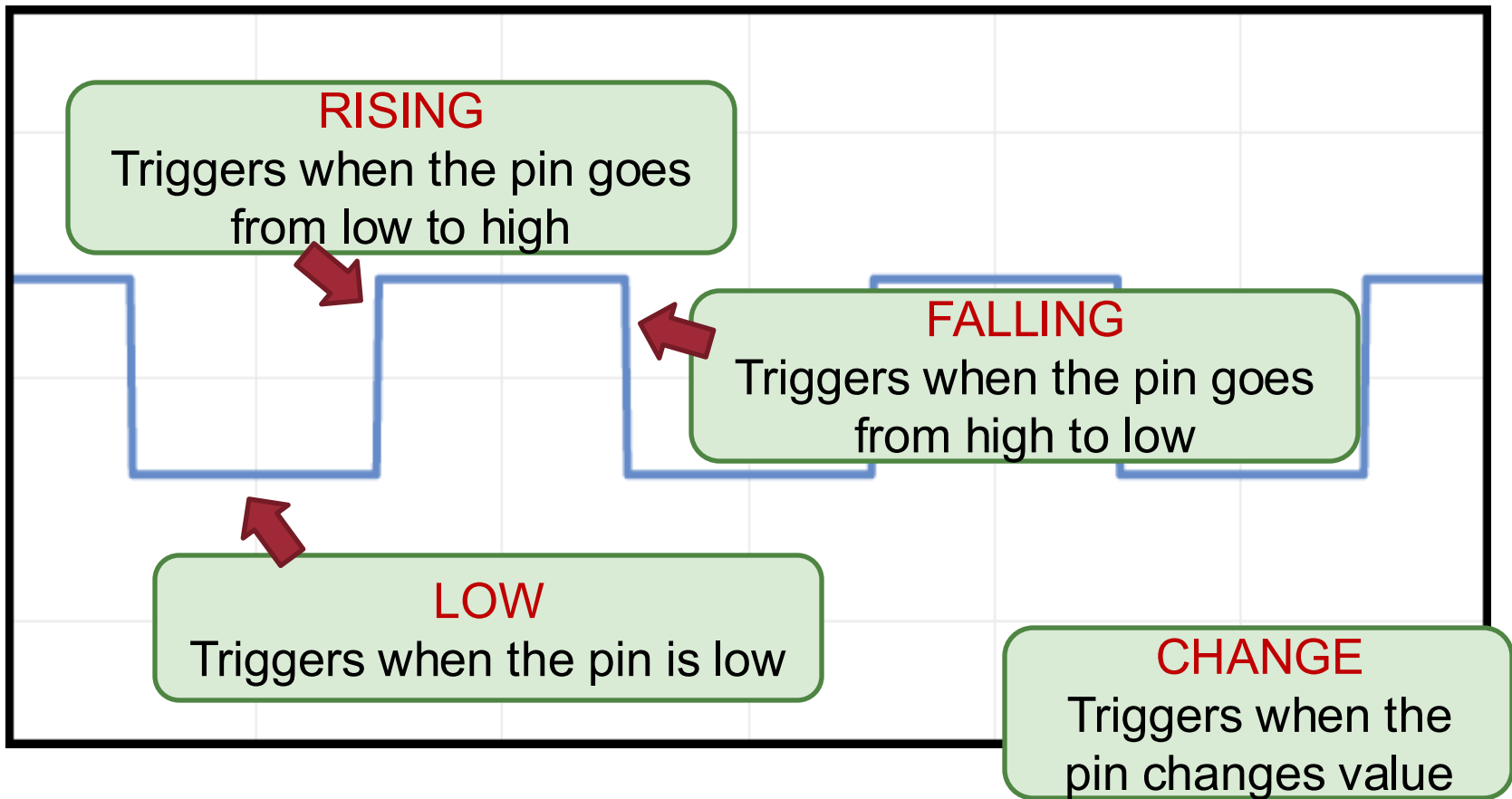
- We are going to use a simple trick to "simulate" a button press / change in input for pin 2
  
- Steps:
  1. Upload the sketch and open the Serial Monitor.
    - Observe the output
  2. Connect a male-to-male wire into pin 2. Leave the other end unconnected.
  3. To simulate a change for pin 2, insert the other end to a ground pin and then quickly remove it.

# Important Functions

- `pinMode( XXX, INPUT_PULLUP)`
  - ❑ inverts the behavior of the pin, where HIGH means the input device is off, and LOW means the input device is on
- `attachInterrupt( IRNum, ISR, mode)`
  - ❑ IRNumber: The interrupt number (NOT the pin number). Use `digitalPinToInterrupt( pin )` to convert easily
  - ❑ ISR: The Interrupt Service Routine. The ISR should be a void `XXXX()` function.
  - ❑ mode: Triggering mode, see next slide

# Four Interrupt **Trigger** Modes

- Triggering Mode is based on the feature of the signal:



# Explanation

- This code demonstrates the scenario where:
  - ❑ the *loop()* is doing some work
  - ❑ need to react to change in pin 2 (e.g. if it is connected to a button)
- How does it work?
  - ❑ Whenever the pin 2 experienced a change, the ISR "blink()" is executed automatically
  - ❑ blink() inverts the state of the LED (high→low, low→high)
  - ❑ the loop() resumes automatically after blink() returns

# Writing ISR: Rules of Thumb

1. Make ISR as short as possible
  - ❑ Common trick: Set a flag / update value in the ISR and let the main loop to do the time consuming part
2. Do not use delay() in ISR
3. Do not use Serial read/write in ISR
4. If a variable is shared between ISR and other code, declare it as **volatile**:
  - ❑ Ensure correct compiler handling
- Caution: When an ISR is executing, all other interrupts have to wait!
  - ❑ May miss important event!

---

# Your Tasks for the Afternoon

- Get your Arduino up and going
- Run through all the examples (time to play).
  - Blink, Serial, Ultrasonic, DC Motor, Interrupts
- Can you use an interrupt to turn on an LED based on the distance measured by the ultrasonic sensor?
- Try out the analog thumbstick! Can you get it working? (requires some googling)
- Pick a sensor from the sensor box and get it working!

---

**END**

---